

The Schema

Schemas in AIKRR can take various forms, depending on the specific knowledge representation formalism being used. Some common types of knowledge representation schemas in AIKRR include:

Semantic Networks: These use nodes and edges to represent concepts and their relationships. Nodes represent entities or concepts, while edges represent relationships between them.

Frames: A frame is a data structure that represents an object or concept along with its attributes and the relationships between those attributes. Frames are used for organizing structured knowledge.

Ontologies: Ontologies define a taxonomy of concepts and their relationships in a specific domain. They often use a hierarchical structure to represent knowledge.

Rules: Rules in AIKRR can be considered a type of schema, as they specify conditions and actions. For example, in rule-based systems, if-then rules are used to represent knowledge and make inferences.

Description Logics: Description logics are a formalism used for knowledge representation in AIKRR. They use a schema to define concepts, roles, and axioms to capture complex relationships and inferences.

Frames: Frames are a way of organizing knowledge by creating structured representations of objects or concepts. Each frame contains slots (attributes) that store information about the object, and these slots can be filled with values.

Schemas in AIKRR play a crucial role in enabling AI systems to understand, reason about, and make decisions based on the knowledge they have. They provide a structured and organized way to represent complex information, which is essential for building intelligent systems capable of performing tasks like natural language understanding, expert systems, and more. The choice of a particular schema or knowledge representation formalism depends on the specific requirements and domain of the AI system being developed.

Let's consider a domain related to animals. We can create a simple schema representing some common attributes and relationships associated with animals:

Entities (Nodes):

Animal

Mammal

Reptile

Bird

Relationships (Edges):

"is a" (for categorization relationships)

"has" (for attribute relationships)

Using this schema, we can create instances for specific animals:

Instance 1:

Animal: Dog

Category: Mammal

Attributes: Fur, Barks, Four legs

Instance 2:

Animal: Crocodile

Category: Reptile

Attributes: Scales, Sharp teeth, Four legs

Instance 3:

Animal: Sparrow

Category: Bird

Attributes: Feathers, Beak, Two legs

In this example, the schema helps organize the knowledge about different types of animals, their categories, and some of their characteristic attributes. This structured representation can be used by AI systems for various tasks, such as classification, inference, and reasoning about animals based on their features and relationships within the schema.

Semantics:

- Semantics are the rules by which we can interpret the sentence in the logic.
- Semantic also involves assigning a meaning to each sentence.

Logical representation can be categorised into mainly two logics:

- a. Propositional Logics
- a. Predicate logics

Advantages of logical representation:

1. Logical representation enables us to do logical reasoning.
2. Logical representation is the basis for the programming languages.

Disadvantages of logical Representation:

1. Logical representations have some restrictions and are challenging to work with.
2. The logical representation technique may not be very natural, and inference may not be so efficient.

Note: Do not be confused with logical representation and logical reasoning as logical representation is a representation language and reasoning is a process of thinking logically.

2. Semantic Network Representation

Semantic networks are alternative of predicate logic for knowledge representation. In Semantic networks, we can represent our knowledge in the form of graphical networks. This network consists of nodes representing objects and arcs which describe the relationship between those objects. Semantic networks can categorize the object in different forms and can also link those objects. Semantic networks are easy to understand and can be easily extended.

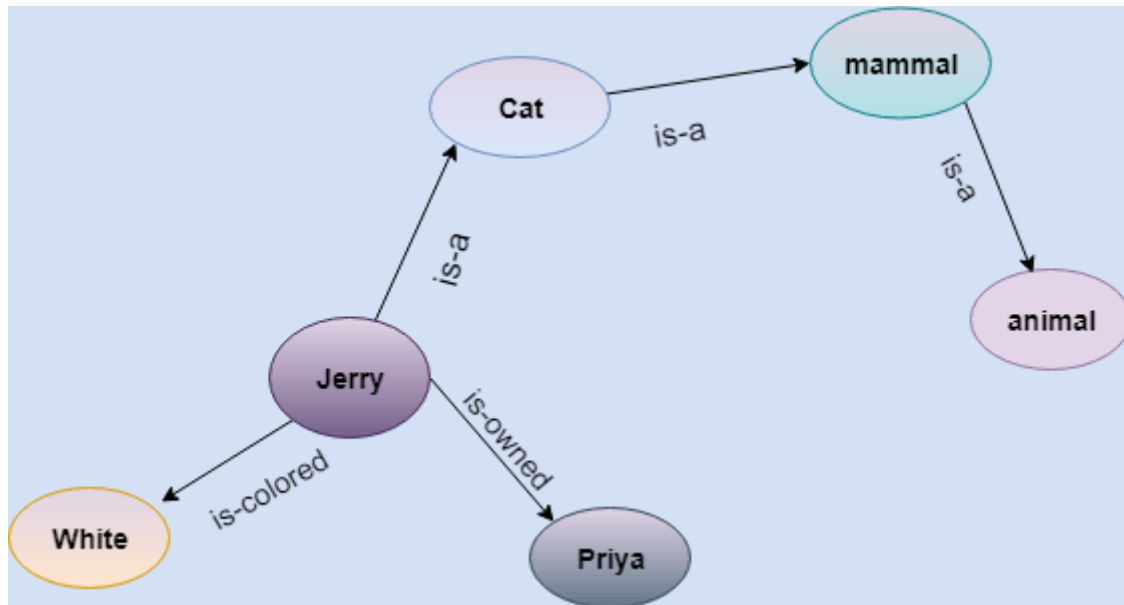
This representation consist of mainly two types of relations:

- a. IS-A relation (Inheritance)
- a. Kind-of-relation

Example: Following are some statements which we need to represent in the form of nodes and arcs.

Statements:

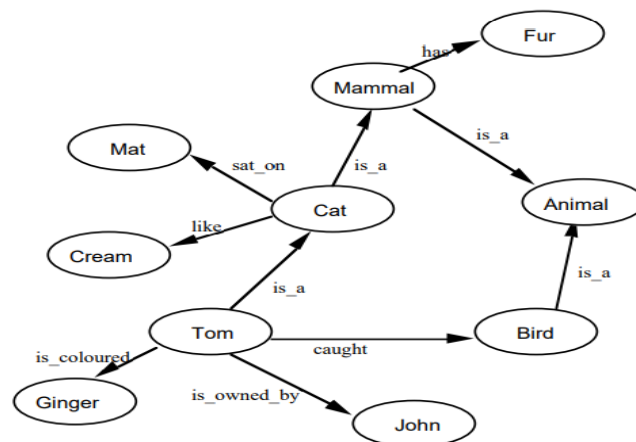
- a. Jerry is a cat.
- a. Jerry is a mammal
- a. Jerry is owned by Priya.
- a. Jerry is White colored.
- a. All Mammals are animal.



In the above diagram, we have represented the different type of knowledge in the fm of nodes and arcs. Each object is connected with another object by some relation.

Semantic Nets

Semantic networks are an alternative to predicate logic as a form of knowledge representation. The idea is that we can store our knowledge in the form of a graph, with nodes representing objects in the world, and arcs representing relationships between those objects. For example, the following:



is intended to represent the data:

- Tom is a cat.*
- Tom caught a bird.*
- Tom is owned by John.*
- Tom is ginger in colour.*
- Cats like cream.*
- The cat sat on the mat.*
- A cat is a mammal.*
- A bird is an animal.*
- All mammals are animals.*
- Mammals have fur.*

Drawbacks in Semantic representation:

1. Semantic networks take more computational time at runtime as we need to traverse the complete network tree to answer some questions. It might be possible in the worst case scenario that after traversing the entire tree, we find that the solution does not exist in this network.
2. Semantic networks try to model human-like memory (Which has 1015 neurons and links) to store the information, but in practice, it is not possible to build such a vast semantic network.
3. These types of representations are inadequate as they do not have any equivalent quantifier, e.g., for all, for some, none, etc.
4. Semantic networks do not have any standard definition for the link names.
5. These networks are not intelligent and depend on the creator of the system.

Advantages of Semantic network:

1. Semantic networks are a natural representation of knowledge.
2. Semantic networks convey meaning in a transparent manner.
3. These networks are simple and easily understandable.

Scripts Goals

a script typically refers to a set of instructions or commands that are executed in a sequential manner. Scripts can serve various purposes in the field of AI, depending on the specific goals and tasks they are designed to accomplish. Some common goals of using scripts in AI include:

- ❖ **Automating Tasks:** Scripts can be used to automate repetitive tasks, such as data preprocessing, model training, or deployment processes. By scripting these tasks, AI practitioners can save time and effort, allowing them to focus on more complex aspects of AI development.
- ❖ **Enhancing Workflow Efficiency:** Scripts can streamline various aspects of AI development, such as data collection, cleaning, and analysis. By creating efficient scripts, AI engineers can accelerate the development process and improve overall workflow efficiency.
- ❖ **Reproducibility and Consistency:** Using scripts ensures that experiments and analyses can be easily reproduced. By documenting and standardizing the procedures in scripts, researchers can maintain consistency across different experiments and analyses, facilitating the validation of results and the comparison of different methodologies.

- ❖ **Facilitating Model Deployment:** Scripts play a crucial role in deploying AI models into production environments. They help in setting up the necessary infrastructure, integrating the model into existing systems, and enabling the model to interact with end-users or other applications seamlessly.
- ❖ **Customizing and Extending Functionality:** AI developers use scripts to customize and extend the functionality of existing AI frameworks and libraries. By writing scripts, developers can incorporate additional features, optimize performance, or adapt AI algorithms to specific use cases and datasets.
- ❖ **Simplifying Complex Processes:** In AI research and development, complex processes such as deep learning model training and hyperparameter tuning can be simplified using scripts. These scripts can automate the optimization process and make it easier for researchers to experiment with different configurations and settings.
- ❖ **Managing Data and Resources:** Scripts are commonly used to manage data pipelines, including data ingestion, transformation, and storage. They also help in resource management, such as handling memory usage, GPU allocation, and distributed computing, which are essential for efficiently handling large-scale AI projects.

Overall, scripts serve as a fundamental tool in the AI development lifecycle, contributing to increased productivity, reproducibility, and the seamless integration of AI technologies into various applications and systems.

Certainly, here are some specific examples of how scripts can be used to achieve various goals in the field of AI:

- ❖ **Data Preprocessing Script:** A script can be written to handle data preprocessing tasks such as data cleaning, normalization, and feature scaling. This ensures that the data is in the appropriate format and is ready for use in training AI models.
- ❖ **Model Training Script:** To automate the model training process, a script can be created to define the model architecture, set up the training parameters, and manage

the training loop. This enables efficient training of complex AI models without manual intervention.

- ❖ **Hyperparameter Optimization Script:** By writing a script for hyperparameter optimization, researchers can automate the process of searching for the optimal set of hyperparameters for a given AI model. This script can implement techniques such as grid search, random search, or Bayesian optimization to fine-tune the model's performance.
- ❖ **Model Evaluation Script:** A script can be used to evaluate the performance of trained AI models by calculating various metrics such as accuracy, precision, recall, and F1 score. This helps in assessing the model's effectiveness and identifying areas for improvement.
- ❖ **Deployment Script:** For deploying AI models into production environments, a deployment script can be written to package the model, set up API endpoints, and handle incoming requests. This script ensures that the AI model can be seamlessly integrated into applications or services.
- ❖ **Data Pipeline Script:** Writing a script for data pipeline management facilitates tasks such as data ingestion from various sources, data transformation, and data storage. This ensures that data flows smoothly through the AI system, from acquisition to analysis and model training.
- ❖ **Resource Management Script:** To efficiently manage computing resources, a script can be created to allocate resources such as GPUs, memory, and CPU cores based on the requirements of different AI tasks. This helps in optimizing resource utilization and reducing overall computational costs.
- ❖ **Natural Language Processing (NLP) Script:** For NLP tasks, scripts can be used for text preprocessing, tokenization, part-of-speech tagging, and sentiment analysis. These scripts help in processing and analyzing textual data, enabling the development of robust NLP applications.

These examples demonstrate how scripts can be applied to various aspects of AI development, from data preprocessing and model training to deployment and resource management, thereby streamlining the development process and improving the overall efficiency of AI systems.

What is a Plan?

For any planning system, we need the **domain description**, **action specification**, and **goal description**. A plan is assumed to be a sequence of actions and each action has its own set of preconditions to be satisfied before performing the action and also some effects which can be positive or negative.

So, we have Forward State Space Planning (FSSP) and Backward State Space Planning (BSSP) at the basic level.

1. Forward State Space Planning (FSSP)

FSSP behaves in a similar fashion like forward state space search. It says that given a start state S in any domain, we perform certain actions required and acquire a new state S' (which includes some new conditions as well) which is called progress and this proceeds until we reach the goal state. The actions have to be applicable in this case.

- **Disadvantage:** Large branching factor
- **Advantage:** Algorithm is Sound

2. Backward State Space Planning (BSSP)

BSSP behaves in a similar fashion like backward state space search. In this, we move from the goal state g towards sub-goal g' that is finding the previous action to be done to achieve that respective goal. This process is called regression (moving back to the previous goal or sub-goal). These sub-goals have to be checked for consistency as well. The actions have to be relevant in this case.

- **Disadvantage:** Not a sound algorithm (sometimes inconsistency can be found)
- **Advantage:** Small branching factor (very small compared to FSSP)

Hence for an efficient planning system, we need to combine the features of FSSP and BSSP which gives rise to Goal Stack planning which will be discussed in the next article.

MOPs

Suppose we were developing an AI program that reasoned about people. One of the first things we'd need to represent is a person, e.g.,

Representation	What it says
john-1: male-person name = "John Smith" age = 22	A male person, named "John Smith," who is 22 years old. This piece of knowledge will be referred to internally by the identifying label john-1 .
event-3: event actor = john-1 action = ingest object = apple-92	An event where John ate an apple. This will be referred to internally by the label event-3 .

This representation was invented and used by many AI researchers, and given the generic name *frame* by Minsky in 1975. The frame format can be used to represent objects, events, abstract relationships, and so on.

Schank's Memory Organization Packets (MOPs) are frames primarily used to represent episodic memory, that is, memories of particular events and their generalizations. For this course, we will use a simple implementation of MOPs and frames that is easy to understand, use, and extend.

The structure of a frame

A frame has (at least) the following parts:

- a name, e.g., **john-1**
- zero or more abstractions (classes to which the concept belongs), e.g., **male-person**
- zero or more *slots* that describe particular properties of the concept, e.g., **name = "John Smith"** and **age = 22**.

The name is for internal cross-reference purposes. Each frame has one name, and there is at most one frame for each name. Names help us keep straight two concepts that otherwise appear identical. For example, we might know about two 22-year-old men named John Smith, **john-1** and **john-2**.

Abstractions of frames

The classes to which a concept belongs are sometimes called the *abstractions* of that concept. Conversely, the members of a class are called its *specializations*.

An abstraction can itself be another frame. For example,

Representation	What it says
male-person: person gender = male	the concept named male-person has the abstraction person and the slot gender = male
ingest-event: event action = ingest	the class of all events involving ingesting

Slots of a frame

The slots of a frame represent particular attributes of the concept being represented, e.g., its age, color, component parts, and so on.

A slot has two parts, a *role* and a *filler*. The role names the attribute, and the filler gives its value.

For example, in the **john-1** frame, **name** and **age** are roles, and **"John Smith"** and **22** are fillers.

Fillers can be the names of frames, e.g., the filler **john-1** in **event-3**.

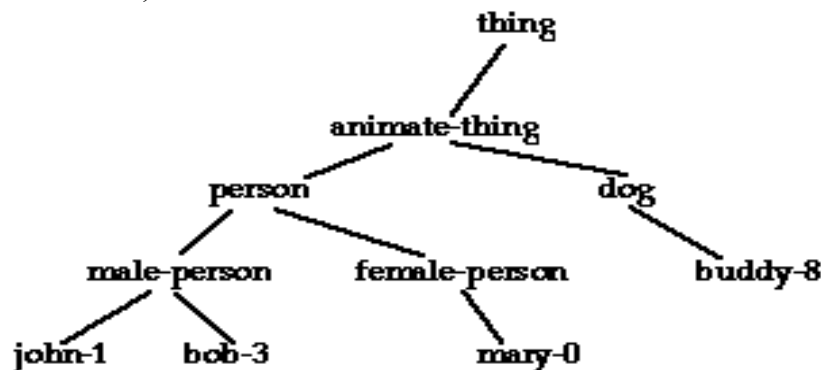
Hierarchies of abstractions

Consider the following set of frames we might want to define, ignoring for the moment the slots in each one:

- **john-1**, a **male-person**
- **male-person**, a **person**
- **person**, an **animate-thing**
- **animate-thing**, a **thing**

In AI, this is called an *abstraction hierarchy*. Another name is *conceptual taxonomy*. Such hierarchies are often drawn graphically, with the most abstract items at the top and the most

specific items at the bottom. For example, the chain of abstractions above might be part of a larger tree of abstractions, like this:



Everything in this tree, except **john-1**, is an abstraction. **john-1** is an *instance* (or *individual*). An instance can not have anything below it in the tree. That is, while we might have an abstraction for "people named John," under which **john-1** and **john-2** might fit, it doesn't make sense to talk about instances of **john-1**. **john-1** is a unique individual. We, most of the AI community, and probably most average human reasoners, ignore such philosophical conundrums as "how can John, age 22, be the same individual as John, age 10, when all the cells in his body have been replaced in those 12 years?"

Immediate abstractions

When we talk about the abstraction of some concept, we will normally mean anything that is above the concept in the abstraction hierarchy. Sometimes, we will talk about the "immediate abstractions" of a concept to refer to those concepts that are exactly one link up the hierarchy.

Inheritance

An important idea related to abstraction hierarchies is *inheritance*. A concept inherits attributes of its abstractions. If elephants are grey, and Clyde is an elephant, then Clyde is grey. If Mary is a person, people are vertebrates, and all vertebrates have bones, then Mary has bones.

Inheritance as default knowledge

There are several different ways to view inherited attributes. In some of the earliest systems, such as Fahlman's NETL, concepts inherited "virtual copies" of the slots on their abstractions. Furthermore, the inherited slot was treated as a default, which could be overridden. Thus, we might have

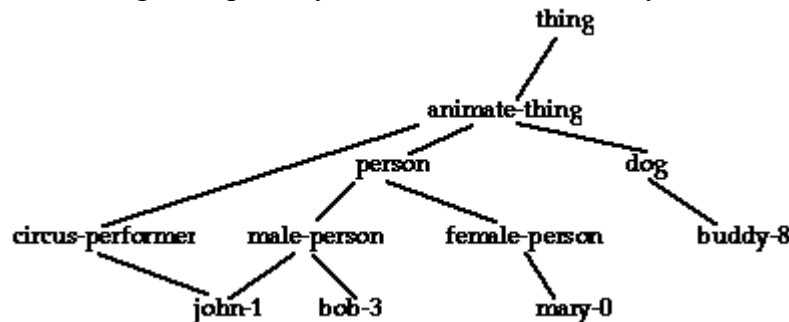
Representation	What it says
elephant: animal color = grey	elephants are normally grey...
clyde-1: elephant color = white	...but Clyde is a white elephant

This interpretation of inherited attributes has problems, however. The first problem is that if fillers can be overridden, then any concept can be put under any other concept. For example,

you could say that a clock is an elephant with all the attributes of elephant overridden appropriately. The second problem arises when you have multiple inheritance.

Multiple inheritance

Multiple inheritance occurs when a concept has more than one immediate abstraction. For example, John might be both a person and a circus performer, where the class of circus performers includes people, elephants, trained fleas, lions, tigers, bears, and several other subclasses of animate things. Graphically, the abstraction hierarchy looks something like this:



The concept **john-1** inherits from both **person** and **circus-performer**. Thus, we might conclude that **john-1** has bones and wears colorful costumes.

But what if the abstractions have conflicting default attributes? The classic conundrum in this area of AI is the Nixon paradox. Nixon was a Republican and a Quaker. Assume that by default Republicans believe in war and Quakers don't. Then what does Nixon believe?

Inheritance as constraint knowledge

An alternative interpretation of inherited attributes is that they are *constraints*. That is, if an abstraction says that elephants are grey, then any instance of an elephant be some shade of grey color.

Constraints can be refined by specializations of an abstraction, but they can't be overridden. Thus, if some elephants are white, we can't say elephants are grey. We might say instead that elephants as a group have some color, then create two subclasses of elephants, white ones and grey ones.

This approach resolves the examples above by not allowing them to be represented in the first place. You can't put the clock class under the elephant class, because the attributes of clocks don't satisfy the constraints of elephants. If Nixon believes in war, then either he is not a Quaker or Quakers don't have that constraint.

An abstraction such as Republican Quakers can be formed, but it will have no members, because nothing can satisfy all the inherited constraints.

The right interpretation of inheritance

The "inheritance of defaults" approach captures an important part of human reasoning. If we hear that Mary went to Europe, we assume, by default, that she flew, because trips to Europe are usually by plane. The "scruffies" of AI, who are concerned with making systems work, take the "defaults" approach. They accept the fact that their systems *could* make clocks a kind of elephant, but believe that reasonable learning algorithms won't do this.

The "inheritance of constraints" approach is appealing to the "neats" of AI, because it avoids paradoxes and lends itself to logical formulation. The hard, i.e., intractable, problem turns out to be determining when one constraint could legally specialize another. This is logically equivalent to theorem proving.

The obvious generalization algorithms, including those used in some classic MOP-based programs, e.g., CYRUS, build constraint-oriented hierarchies. This is because new abstractions are formed by taking features common to some set of instances.

Description logic

Description logic is a formalism used in AI applications for representing and reasoning about the properties of concepts and individuals. It is closely related to first-order logic, but with a more restricted syntax that makes it more suitable for automated reasoning.

Description logic is used in a variety of AI applications, including knowledge representation, natural language processing, and machine learning. In knowledge representation, description logic can be used to represent the domain knowledge of an expert in a machine-readable form. This knowledge can then be used by AI applications to answer questions or make decisions.

In natural language processing, description logic can be used to represent the meaning of sentences in a machine-readable form. This can be used to automatically generate answers to questions, or to translate sentences from one language to another.

In machine learning, description logic can be used to represent the background knowledge of a domain. This knowledge can be used to improve the performance of machine learning algorithms.

Description Logic in AI

Why Description Logics ?

If FOL is directly used without some kind of restriction, then

- The structure of the knowledge/information is lost (no variables, concepts as classes, and roles as properties),
- The expressive power of FOL is too high for having good (computational properties and efficient procedures).

Description logics are a family of logics concerned with knowledge representation.

A **description logic** is a decidable fragment of first-order logic, associated with a set of automatic reasoning procedures.

Axioms, Disjunctions and Negations

In Description Logics and Knowledge Representation and Reasoning (AIKRR), axioms, disjunctions, and negations are fundamental elements used to model and reason about knowledge in a formal and logical way.

Axioms:

Axioms are statements that define relationships, constraints, or rules within a knowledge base. They are used to assert fundamental properties and facts about the concepts, individuals, and roles in the domain.

Example Axiom:

$\text{Person} \sqsubseteq \text{Animal}$ (Every person is an animal.)

Explanation: This axiom asserts that the concept of a person is a subset of the concept of an animal, capturing the relationship that all people are a type of animal.

Disjunctions:

Disjunctions allow for the expression of alternative possibilities. In DL, the "or" operator ($\sqcup$) is used to represent disjunctions between concepts.

$\text{Individual} \sqsubseteq \text{Vegetarian} \sqcup \text{Carnivore}$ (An individual can be either a vegetarian or a carnivore.)

This disjunction allows for individuals to be classified as either vegetarians or carnivores, providing flexibility in representing diverse characteristics.

Negations:

Explanation: Negations are used to express the absence or complement of a concept. In DL, the "not" operator ($\neg$) is used to represent negations.

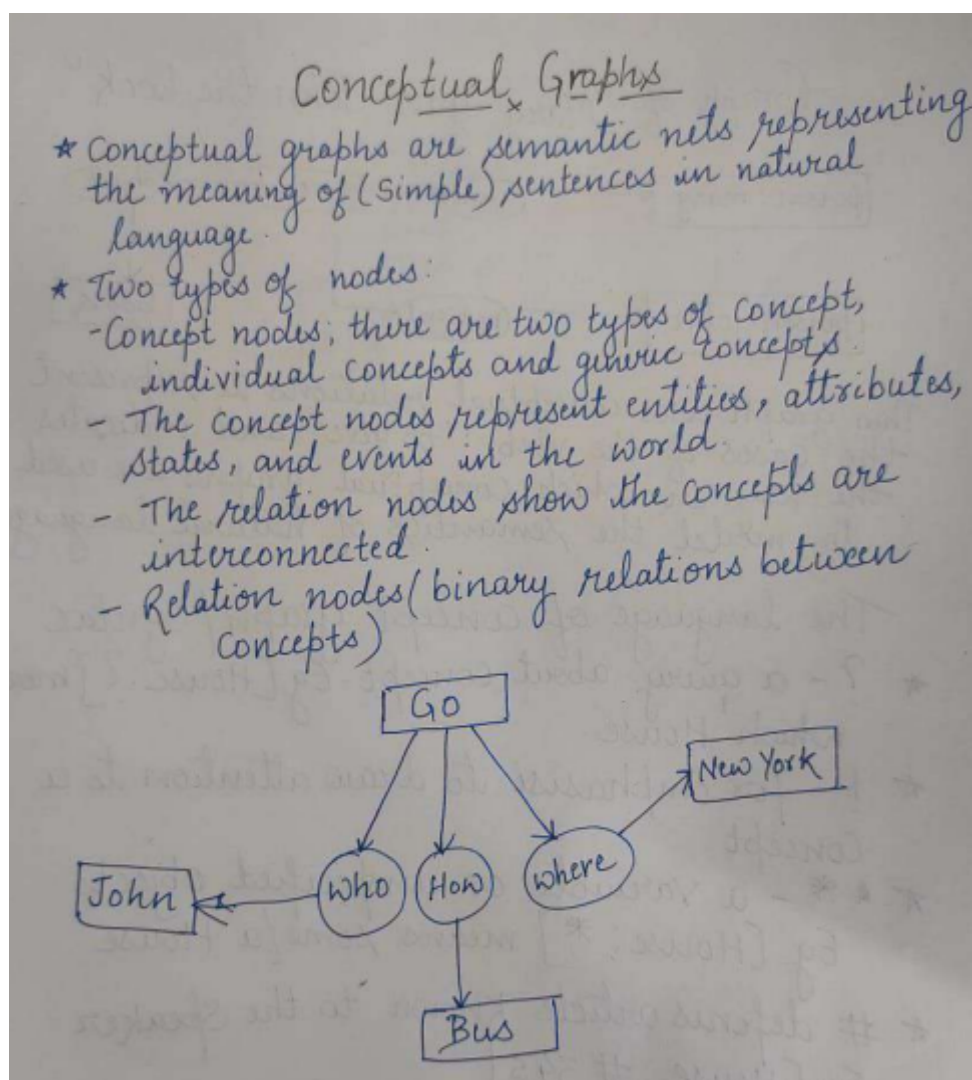
Example Negation:

$\neg \text{Individual} \sqcap \text{Herbivore}$ (An individual is not a herbivore.)

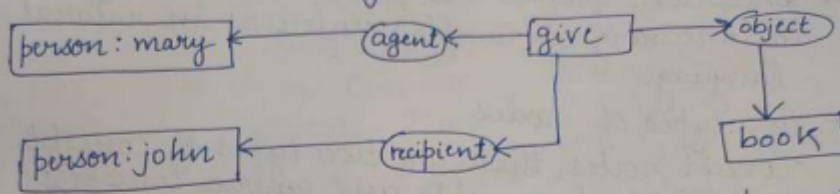
This axiom asserts that there is no individual that falls under both the concept of "individual" and "herbivore".

In combination, these elements enable the expression of complex knowledge structures. For instance, you could have an axiom that states

$\text{Vegetarian} \sqsubseteq \neg \text{Carnivore}$, meaning that all vegetarians are not carnivores. Such axioms, disjunctions, and negations contribute to the expressive power of Description Logics, allowing for the formal representation and reasoning about diverse domains in AIKRR.



Graph of "Mary gave John the book"



This graph uses conceptual relations to represent the cases of the verb "to give" and indicates the way in which conceptual graphs are used to model the semantics of natural language.

The language of concept graph/syntax

- * ? - a query about concept. Eg [House: ?] means which House.
- * ! - for emphasis to draw attention to a concept
- * * - a variable or unspecified objects
Eg [House: *] means some/a House.
- * # definite article known to the speaker.
Eg [House: # 345]
- * @ - quantification. Eg [House: @n] means 'n' Houses

Inference

- * Generalizing and specializing graphs operations to create new graphs from existing graphs.
 - * Copy: for a new graph exactly copied.
 - * Restrict: Replace nodes by a node representing their specification.
 - * Join: Combine two graphs into a simple graph. This is a special restriction.
 - * Simplify: delete duplicate relations.